{EPITECH}

ALICE'S ADVENTURES IN WONDERLAND_

< BOOTSTRAP />

ALICE'S ADVENTURES IN WONDERLAND

Manage and parse CLI options

Create a script named `calculator.py` that performs **basic arithmetic operations** using **CLI arguments**. Your script use **the `argparse` module** and must:

- ✓ take exactly two numeric arguments and support the following options:
  - `--add` for addition;
  - `--sub` for subtraction;
  - `--mul` for multiplication;
  - `--div` for division with options `--float` (resp. `--int`) for float (resp. integer) division;
- ✓ display the operation and its result (as a float) in the terminal;
- ✓ handles invalid input and errors gracefully.

Some examples of usage:

```
./calculator.py --sub 42 -4.0
42.0 - -4.0 = 46.0

./calculator.py --mul 42 -4.0
42.0 × -4.0 = -168.0

./calculator.py --div 42 -4.0
Info: No division mode specified. Using float division by default.
42.0 / -4.0 = -10.5

./calculator.py --div --int 42 -4.0
42.0 // -4.0 = -11

./calculator.py --div --float 42 -4.0
42.0 / -4.0 = -10.5

./calculator.py --div --int --float 42 -4.0
Warning: Both --int and --float provided. Using float division by default.
42.0 / -4.0 = -10.5

./calculator.py --div --float 42 0
Error: Division by zero is not allowed.

./calculator.py --sqrt 42 -4.0
usage: bootstrap.py [-h] (--add | --sub | --mul | --div) [--float] [--int] [numbers ...]
calculator.py: error: one of the arguments --add --sub --mul --div is required

./calculator.py --add 42 -4.0 666
Error: You must provide exactly two numbers.
```


Tooling for NLP tasks

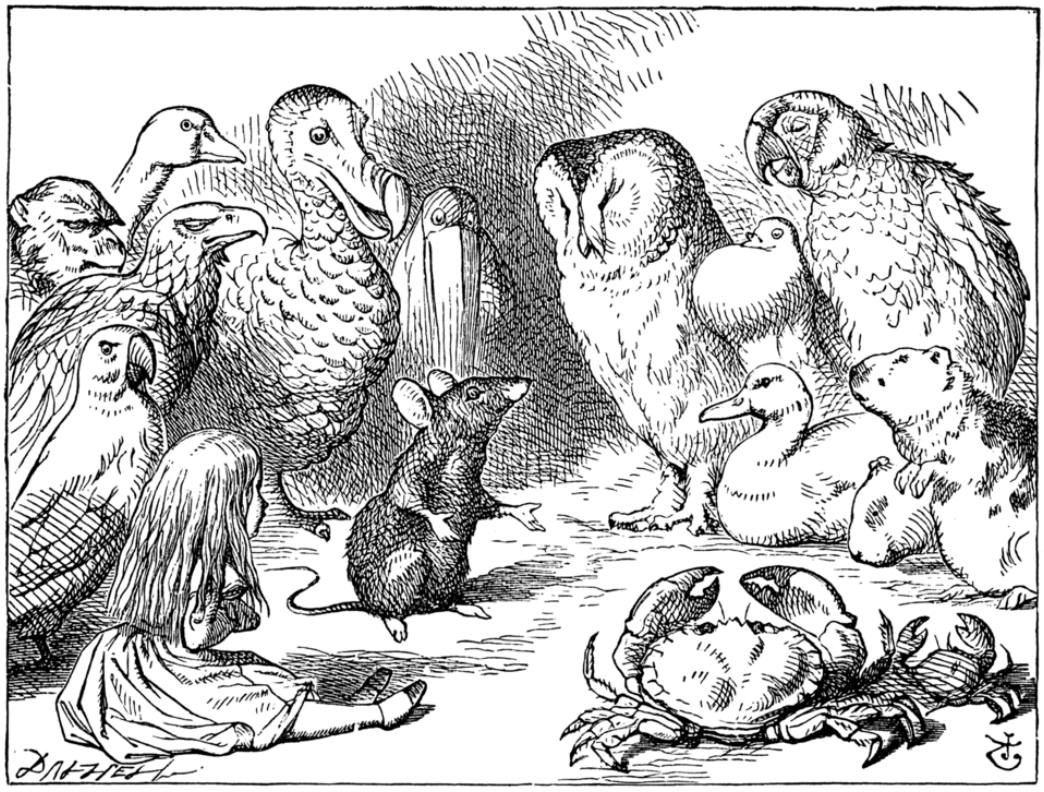
Every job requires the right tools to be done effectively. NLP is no exception.

Downloader

First of, visit Project Gutenberg, download manually any book, and take a peek at it. Then, find and download the **CSV offline catalog**. Eventually, write a program `tools.py` which is able to:

- ✓ return a dictionary with book information (*id*, *title*, *authors* and *bookshelves* as strings) when using the `--info <ID>` option, where `<ID>` is an int representing the book's id;
- ✓ download and save the book in Plain Text UTF-8 format when using the `--download <ID>` option.

```
Terminal
$> ./tools.py --info 11
{'id': '11', 'title': "Alice's Adventures in Wonderland", 'authors': 'Carroll, Lewis, 1832-1898',
'bookshelves': "Children's Literature; Category: Children & Young Adult Reading; Category: Novels;
Category: Classics of Literature; Category: British Literature"}
```



Cleaner

Let's add another feature in order to clean a text, such as the content of the book file.
The `--clean <TEXT>` option cleans a text (provided as a string via CLI), and must at least:

- ✓ remove Project Gutenberg's header and footer;
- ✓ replace multiple consecutive spaces or tabulations with one space;
- ✓ lowercase the text only if the `--lower` flag is provided.

Eventually, it returns the clean content as a string.

```
Terminal
$> ./tools.py --clean "Oh dear!      Oh dear!      I shall be late" --lower
oh dear! oh dear! i shall be late
```

Tokenizer

The `--tokenize <TEXT>` option must split a text (provided as a string via CLI):

- ✓ into words by default;
- ✓ into sentences if the flag `--sent` is provided.

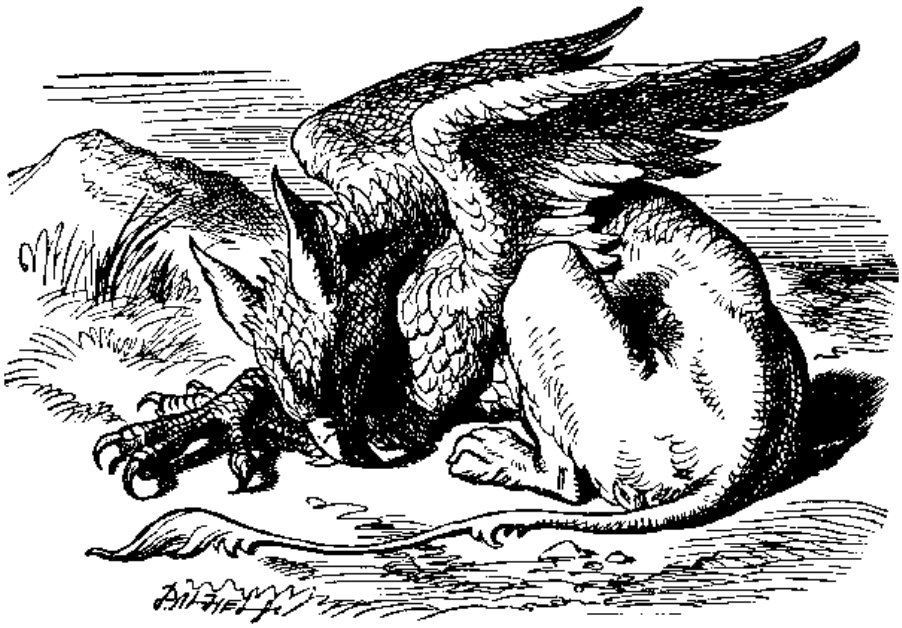
After the tokenization, it ”cleans” the result by removing:

- ✓ the tokens which are stopwords, if the flag `--stop` is provided;
- ✓ the tokens which are punctuation symbols, if the flag `--punct` is provided.

Eventually, it returns the tokens as a list of strings.

```
Terminal
$> ./tools.py --tokenize "Alice had begun to think that very few things indeed were really
impossible." --sent
['Alice had begun to think that very few things indeed were really impossible.']
```

```
Terminal
$> ./tools.py --tokenize "Alice had begun to think that very few things indeed were really
impossible." --punct --stop
['Alice', 'begun', 'think', 'things', 'indeed', 'really', 'impossible']
```



POS tagger

The `--postag <TOKENS>` option assigns part-of-speech tags to a list of tokens (separated by spaces).
It returns a list of tuples, such as `[(<TOKEN>, <TAG>), (<TOKEN>, <TAG>), ...]`.

Terminal

```
$> ./tools.py --postag "The Caterpillar was the first to speak. What size do you want to be? it
asked."
[('The', 'DT'), ('Caterpillar', 'NNP'), ('was', 'VBD'), ('the', 'DT'), ('first', 'JJ'), ('to',
'TO'), ('speak.', 'VB'), ('What', 'WP'), ('size', 'NN'), ('do', 'VBP'), ('you', 'PRP'), ('want',
'VB'), ('to', 'TO'), ('it', 'PRP'), ('asked.', 'VB')]
```

Tokens normalizer

The `--normalize <TOKENS>` option reduces a list of tokens (separated by spaces) to their canonical form, using *lemmatization* by default. If the flag `--stem` is provided, it performs *stemming* instead. It returns a list of normalized tokens, such as `[<TOKEN>, <TOKEN>, <TOKEN>, ...]`.

Make some research about the differences, pros, and cons of each of these methods.
You should find a way to increase the accuracy of your lemmatization method.

Terminal

```
$> ./tools.py --normalize "The game is going on rather better now" --stem
['the', 'game', 'is', 'go', 'on', 'rather', 'better', 'now']
```

Terminal

```
$> ./tools.py --normalize "The game is going on rather better now"
['The', 'game', 'be', 'go', 'on', 'rather', 'good', 'now']
```

v 0.2.1